

ALLIANZ UK · AGENT BRICKS ON DATABRICKS

# Garage Repair Checker

---

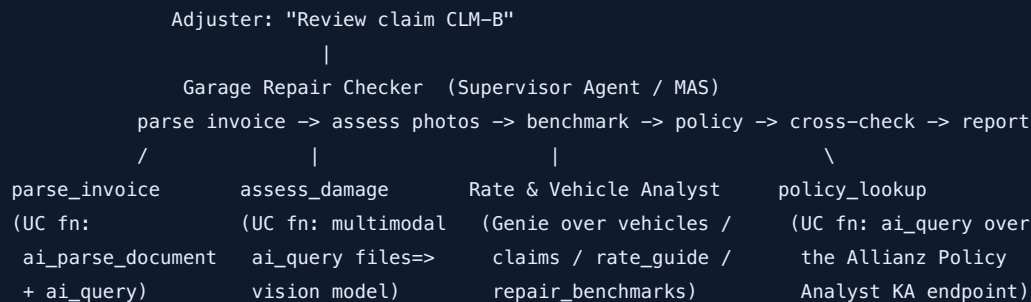
An Agent Bricks supervisor that reviews pre-repair photos and garage invoices to flag inflated repairs, AI-generated imagery, and early total losses - before any repair work begins.

How the second Agent Bricks demo was built on Databricks, end to end, so the team can recreate or extend it.

The demo is an **Agent Bricks Supervisor Agent** that reviews a motor claim's pre-repair photos and the garage's repair invoice, cross-checks them against a standard rate guide and the real Allianz Motor policy, and decides whether the repair is justified, whether the invoice should be challenged, whether the submitted imagery can be trusted, and whether the vehicle should be treated as a total loss. It adds **live multimodal** photo analysis on top of the pattern established by the Coverage Gap Advisor.

All assets are in `mlops_pj.garage_checker` on the `DEFAULT` (Azure) workspace.

## 1. Architecture



Mapping to the use-case brief: - "Extract and itemise repair costs from invoice PDFs" -> `parse_invoice` (live `ai_parse_document` + `ai_query`). - "Cross-reference invoice line items against visible damage in photos" -> supervisor cross-check using `assess_damage`. - "Benchmark labour hours and parts costs" -> Genie over the `rate_guide` / `repair_benchmarks` tables. - "AI-generated image detection" -> `assess_damage` returns an authenticity verdict; `claims.photo_exif_status` adds the metadata signal. - "Total loss identification (Phase 2)" -> supervisor computes repair-to-ACV vs the total-loss threshold and routes to a total-loss adjuster. - "Policy document (real DOI)" -> `policy_lookup` over the existing Allianz Policy Analyst Knowledge Assistant (real Motor DOI).

## 2. Prerequisites

- Databricks CLI authenticated to the workspace (profile `DEFAULT`).
- A running serverless SQL warehouse (we used `ec3b3b85811ecd2c`). `ai_parse_document` requires DBR/serverless 17.1+ - the warehouse here supports it.
- Python with `databricks-sdk`; the Supervisor Agent API needs `>= 0.103` (we use 0.114.0 via `PYTHONPATH=./agentlibs`).
- Foundation Model API access: `databricks-claude-sonnet-4-5` is used for invoice parsing AND as the vision model for photo analysis (it supports multimodal `files =>`). `databricks-llama-4-maverick` is a vision fallback.
- `weasyprint` locally to render the invoice PDFs.
- The existing Allianz Policy Analyst Knowledge Assistant endpoint `ka-924e4b7c-endpoint` (from the Coverage Gap build) - it already indexes the real Allianz Motor DOI.

Note on multimodal: `ai_query(..., files => content)` reads JPEG/PNG bytes straight from a binary column - no upload/encoding step. The `responseFormat => 'json_object'` option is NOT supported on the Claude endpoint, so `parse_invoice` asks for raw JSON in the prompt instead.

### 3. Schema, volumes, and mocked assets

```
databricks schemas create garage_checker mlops_pj -p DEFAULT --comment "Allianz Garage Repair Checker POV"
databricks volumes create mlops_pj garage_checker photos MANAGED -p DEFAULT
databricks volumes create mlops_pj garage_checker invoices MANAGED -p DEFAULT
```

#### 3a. Pre-repair photos (mocked, AI-generated)

Five photos in `assets/photos/`, generated with an image model and prompt-tuned per scenario: - `clm_a_golf_front.png` - VW Golf, isolated front O/S bumper + headlamp damage (honest claim). - `clm_b_focus_wing.png` / `clm_b_focus_wide.png` - Ford Focus, a single damaged N/S front wing; the wide shot proves the rest of the car is pristine (sets up the panel-stuffing fraud). - `clm_c_synthetic.png` - deliberately artefacted/uncanny image (warped wheel, melted trim, impossible reflections) to trigger the synthetic-image detector. - `clm_d_audi_totalled.png` - Audi A4 with severe frontal collision damage (total-loss candidate).

```
# upload
for f in assets/photos/*.png; do databricks fs cp "$f" "dbfs:/Volumes/mlops_pj/garage_checker/photos/${basename "$f"}" -p D
```

#### 3b. Repair invoices (mocked PDFs)

`scripts/gen_invoices.py` renders one PDF per claim with WeasyPrint, using three different garage "brands"/layouts so the parser is shown to be format-robust. Fraud signals are embedded in the data (labour above book time, an inflated hourly rate, OEM parts where aftermarket is allowed, and panels billed that do not appear in the photos).

```
python3 scripts/gen_invoices.py
for f in assets/invoices/*.pdf; do databricks fs cp "$f" "dbfs:/Volumes/mlops_pj/garage_checker/invoices/${basename "$f"}" -p D
```

| Claim | Vehicle         | Invoice total (inc VAT) | ACV        | Intended verdict              |
|-------|-----------------|-------------------------|------------|-------------------------------|
| CLM-A | 2021 VW Golf    | GBP 883.80              | GBP 15,500 | Approve                       |
| CLM-B | 2019 Ford Focus | GBP 4,185.60            | GBP 9,800  | Flag - panel-stuffing leakage |
| CLM-C | 2020 BMW 320i   | GBP 3,084.96            | GBP 18,000 | Flag - synthetic imagery      |
| CLM-D | 2015 Audi A4    | GBP 10,500.00           | GBP 8,500  | Total loss (~124% of ACV)     |

### 4. Structured data

`sql/garage/01_create_garage_data.sql` creates (run via `sql/garage/run_sql_garage.py`): - `vehicles` - registration, make/model, year, mileage, and **ACV (pre-accident value)** in GBP. - `claims` - FNOL + submission metadata, including `reported_damage_area`, `photo_exif_status` (the image-metadata signal), `garage_name` and `invoice_total_gbp` (inc VAT). - `rate_guide` - per operation: standard `book_hours`, expected labour rate, and OEM vs aftermarket part prices. - `repair_benchmarks` - standard labour rates and the **total\_loss\_threshold** (60% of ACV). - `claim_photos` - BINARY photo content keyed by `claim_id` (loaded from the photos volume; claim id derived from the filename). Used by `assess_damage`. - `invoice_files` - BINARY invoice PDF content keyed by `claim_id`. Used by `parse_invoice`.

```
python3 sql/garage/run_sql_garage.py sql/garage/01_create_garage_data.sql
```

## 5. The live functions

### 5a. `parse_invoice(claim_id)` - `sql/garage/02_parse_invoice.sql`

Reads the claim's invoice PDF from `invoice_files`, parses it with `ai_parse_document`, then itemises it into clean JSON (garage, line items by section, hours, prices, subtotal/VAT/total) with `ai_query`. Live at agent-call time.

```
SELECT mlops_pj.garage_checker.parse_invoice('CLM-B');
```

### 5b. `assess_damage(claim_id)` - `sql/garage/03_assess_damage.sql`

The multimodal core. For each photo of the claim it runs `ai_query('databricks-claude-sonnet-4-5', '<assessor + forensics prompt>', files => content)` and concatenates the results. Returns, per photo: visible damage by panel, overall severity, and an authenticity verdict ( `AUTHENTICITY: GENUINE` or `SUSPECTED SYNTHETIC - FLAG FOR REVIEW` ).

```
SELECT mlops_pj.garage_checker.assess_damage('CLM-C'); -- flags the synthetic image
```

Gotcha: `ai_query` cannot appear inside a `collect_list` aggregate (SQLSTATE 42845). Compute the per-photo `ai_query` in an inner subquery, then aggregate with `collect_list` / `concat_ws` in the outer query.

### 5c. `policy_lookup(question)` - `sql/garage/04_create_policy_lookup.sql`

`ai_query` over the existing `ka-924e4b7c-endpoint` (Allianz Policy Analyst KA). Confirms motor terms - aftermarket vs genuine parts, betterment, excess, total-loss provisions - and returns citable wording. Same reliable pattern as the Coverage Gap demo (we use a UC function, not the native KA subagent).

```
python3 sql/garage/run_sql_garage.py sql/garage/02_parse_invoice.sql
python3 sql/garage/run_sql_garage.py sql/garage/03_assess_damage.sql
python3 sql/garage/run_sql_garage.py sql/garage/04_create_policy_lookup.sql
```

## 6. Genie Space (Rate & Vehicle Analyst)

`agents/garage/create_genie_garage.py` builds a Genie space over `vehicles`, `claims`, `rate_guide`, and `repair_benchmarks` with instructions on computing the repair-to-ACV ratio and the total-loss threshold.

```
python3 agents/garage/create_genie_garage.py
```

## 7. Supervisor Agent (Garage Repair Checker)

`agents/garage/create_mas_garage.py` creates the supervisor, wires the four tools, and adds four worked examples (clean, panel-stuffing, synthetic, total loss). The instructions encode the workflow: parse invoice -> assess photos -> benchmark + ACV via Genie -> confirm policy terms -> cross-check -> structured report, with the total-loss ratio computed for every claim.

```
PYTHONPATH="./.agentlibs" python3 agents/garage/create_mas_garage.py
```

Test end to end:

```
python3 agents/garage/test_garage.py "Review claim CLM-B and tell me if the repair invoice is justified."
```

## 8. Resource identifiers (this build)

| Component                           | Identifier                                                                                              |
|-------------------------------------|---------------------------------------------------------------------------------------------------------|
| Schema / volumes                    | <code>mlops_pj.garage_checker</code> , volumes <code>photos</code> and <code>invoices</code>            |
| UC Function (invoice parse)         | <code>mlops_pj.garage_checker.parse_invoice</code>                                                      |
| UC Function (damage + authenticity) | <code>mlops_pj.garage_checker.assess_damage</code> (vision: <code>databricks-claude-sonnet-4-5</code> ) |
| UC Function (policy lookup)         | <code>mlops_pj.garage_checker.policy_lookup</code> (ai_query over <code>ka-924e4b7c-endpoint</code> )   |
| Genie Space                         | <code>01f15edd7b931af0a5e4fde730e3f0de</code> (Allianz Repair Rate & Vehicle Analyst)                   |
| Supervisor Agent                    | MAS id <code>08625583-a68a-4838-9dc6-2ccf2ac1c5b1</code> , endpoint <code>mas-08625583-endpoint</code>  |

## 9. Troubleshooting

- `responseFormat type json_object is not supported for this model` : the Claude endpoint rejects `responseFormat` . Ask for raw JSON in the prompt (as `parse_invoice` does).
- `... should not appear in the arguments of an aggregate function` (42845): don't call `ai_query` inside `collect_list` ; wrap it in an inner subquery first (see 5b).
- `ai_parse_document` not found: needs serverless/DBR 17.1+ - confirm the warehouse.
- Vision model returns nothing: confirm the photo loaded into `claim_photos` (the claim id is derived from the filename prefix `clm_a_` -> `CLM-A` ).
- Genie can't resolve a claim: use the exact claim id (e.g. `CLM-B` ).
- Supervisor endpoint slow on first call: it makes 4+ tool calls (PDF parse + vision on each photo + Genie + KA); a full run is ~3-4 minutes. Warm it before the demo.

## Demo assets - mocked pre-repair photos

---



CLM-A - VW Golf: isolated front bumper + headlamp damage (clean claim, approve).



CLM-B - Ford Focus: single damaged near-side wing.



CLM-B - the rest of the car is pristine (sets up panel-stuffing detection).



CLM-C - deliberately AI-generated image (warped wheel, melted trim) - flagged as synthetic.



CLM-D - Audi A4: severe frontal collision (total-loss candidate).